9 - Projeto Orientado a Objetos e Linguagens de Programação de Alto Nível

No Capítulo 1, examinamos como as camadas de linguagens foram construídas ao longo do tempo em torno do hardware para facilitar a computação para o programador de aplicações. No Capítulo 6, examinamos o código de máquina e, em seguida, uma linguagem de montagem que permite ao programador usar mnemônicos para representar instruções em vez de números.

As linguagens de montagem (Assembly) são um passo na direção certa, mas o programador ainda precisa pensar em termos de instruções de máquina individuais. Para superar esse obstáculo, introduzimos o pseudocódigo como uma maneira informal de descrever algoritmos; o pseudocódigo está mais próximo de como os humanos pensam e se comunicam. As linguagens de programação de alto nível são uma maneira muito formal de realizar a mesma coisa. Como os computadores podem executar apenas código de máquina, tradutores foram desenvolvidos para traduzir programas escritos nessas linguagens de alto nível para código de máquina.

OBJETIVOS

Após estudar este capítulo, você deverá ser capaz de:

- distinguir entre design funcional e design orientado a objetos.
- descrever as etapas do processo de design orientado a objetos.
- aplicar o processo de design orientado a objetos.
- nomear, descrever e dar exemplos dos três ingredientes essenciais de uma linguagem orientada a objetos.
- descrever o processo de tradução e distinguir entre montagem, compilação, interpretação e execução.
- nomear quatro paradigmas de programação distintos e apresentar uma linguagem característica de cada um.
- definir os conceitos de tipo de dado e tipagem forte.
- compreender como as construções de design top-down e design orientado a objetos são implementadas em linguagens de programação.

Antes de analisarmos as linguagens de alto nível, faremos uma digressão para analisar o projeto (ou design) orientado a objetos. O projeto orientado a objetos é outra maneira de analisar o processo de design, que considera um programa do ponto de vista dos dados em vez de tarefas. Como a funcionalidade associada a esse processo de design é frequentemente incorporada em linguagens de programação de alto nível, precisamos entender esse processo de design antes de analisar linguagens de alto nível específicas.

9.1 Metodologia Orientada a Objetos

Abordamos o design top-down primeiro porque ele espelha mais de perto a maneira como os humanos resolvem problemas. Como você viu anteriormente neste livro, uma solução top-down produz uma hierarquia de tarefas. Cada tarefa ou ação nomeada opera nos dados passados a ela por meio de sua lista de parâmetros para produzir a saída desejada. As tarefas são o foco de um design top-down. O design orientado a objetos, por outro lado, é uma metodologia de resolução de problemas que produz uma solução para um problema em termos de entidades autocontidas chamadas **objetos**, que são compostas por dados e operações que manipulam os dados. O design orientado a objetos concentra-se nos objetos e em suas interações dentro de um problema. Uma vez que todos os objetos dentro do problema são reunidos, eles constituem a solução para o problema.

Nesse processo, os princípios de resolução de problemas de Polya são aplicados aos dados e não às tarefas.

Orientação a Objetos

Os dados e os algoritmos que os manipulam são agrupados na visão orientada a objetos, tornando cada objeto responsável por sua própria manipulação (comportamento). Subjacente ao projeto orientado a objetos (POO) estão os conceitos de **classes** e **objetos**.

Um objeto é uma coisa ou entidade que faz sentido dentro do contexto do problema. Por exemplo, se o problema estiver relacionado a informações sobre alunos, em tese um aluno seria um objeto na solução. Um

grupo de objetos semelhantes é descrito por uma classe de objetos (ou classe, para abreviar). Embora não haja dois alunos idênticos, eles têm propriedades (dados) e comportamentos (ações) em comum. Alunos são humanos, homens ou mulheres, que frequentam cursos em uma escola (pelo menos na maior parte do tempo). Portanto, alunos seriam uma classe. A palavra **classe** refere-se à ideia de classificar objetos em grupos relacionados e descrever suas características comuns. Ou seja, uma classe descreve as propriedades e os comportamentos que os objetos da classe exibem. Qualquer objeto específico é uma **instância** (exemplo concreto) da classe.

A resolução de problemas orientada a objetos envolve o isolamento das classes dentro do problema. Os objetos se comunicam entre si enviando mensagens (invocando os subprogramas uns dos outros). Uma classe contém campos que representam as propriedades e os comportamentos da classe. Um campo pode conter valores de dados e/ou métodos (subprogramas). Um **método** é um algoritmo nomeado que manipula os valores de dados no objeto. Uma classe, no sentido geral, é um padrão para a aparência de um objeto (dados) e como ele se comporta (métodos).

? It's in! No, it's out! (Está dentro! Não, está fora!)

Você já assistiu a uma partida de tênis na televisão em que um jogador pede a confirmação de uma chamada de linha? Em uma tela grande, você pode ver a trajetória da bola e o ponto de impacto, que indica se ela foi dentro ou fora. Como eles fazem isso? Por computador, é claro. Um sistema usa quatro câmeras digitais de alta velocidade com software que rastreia uma bola, determina sua trajetória e mapeia seu ponto de impacto. Essas câmeras são conectadas entre si e a um computador principal usando tecnologia sem fio.

* Objeto: Uma entidade ou algo que é relevante no contexto de um problema.
* Classe de objeto (classe): Uma descrição de um grupo de objetos com propriedades e comportamentos semelhantes.
* Campos: Itens nomeados em uma classe; podem ser dados ou subprogramas.
* Método: Um algoritmo nomeado que define um aspecto do comportamento de uma classe.

Metodologia de Projeto

O processo de decomposição que apresentamos envolve quatro etapas. O **brainstorming** (tempestarde de idéias) é a etapa na qual fazemos uma primeira tentativa de determinar as classes do problema. A **filtering** (filtragem) é a etapa na qual revisamos as classes propostas determinadas na etapa de brainstorming para ver se alguma pode ser combinada ou se alguma está faltando. Cada classe que sobrevive à etapa de filtragem é examinada com mais cuidado na próxima etapa.

Os **scenarios** (cenários) é a etapa em que o comportamento de cada classe é determinado. Como cada classe é responsável por seu próprio comportamento, chamamos esses comportamentos de **responsabilidades**. Nesta etapa, perguntas do tipo "e se" são exploradas para garantir que todas as situações sejam examinadas. Quando todas as responsabilidades de cada classe são determinadas, elas são registradas, juntamente com os nomes de quaisquer outras classes com as quais a classe deve colaborar (interagir) para concluir sua responsabilidade.

A **responsibility** (responsabilidade) é a última etapa, na qual os algoritmos são escritos para as responsabilidades de cada uma das classes. Um dispositivo de notação chamado cartão CRC é uma maneira prática de registrar as informações sobre cada classe nesta etapa.

Vamos analisar cada uma dessas etapas com mais detalhes.

Brainstorming

O que é brainstorming? O dicionário define-o como uma técnica de resolução de problemas em grupo que envolve a contribuição espontânea de ideias de todos os membros do grupo.[1] Brainstorming traz à mente um filme ou programa de TV em que um grupo de jovens brilhantes discute ideias sobre um slogan publicitário para o mais recente produto revolucionário. Essa imagem parece estar em desacordo com a imagem tradicional de um analista de computadores trabalhando sozinho em um escritório fechado e sem janelas por dias e que finalmente se levanta gritando: "Ah, há!". À medida que os computadores se tornaram mais poderosos, os problemas que podem ser resolvidos tornaram-se mais complexos, e a imagem do gênio trancado em uma sala sem janelas tornou-se obsoleta. Soluções para problemas complexos precisam de soluções novas e inovadoras baseadas em "Ah-há!" coletivos – não o trabalho de uma única pessoa.

No contexto da resolução de problemas orientados a objetos, o brainstorming é uma atividade em grupo projetada para produzir uma lista de possíveis classes a serem usadas para resolver um problema específico. Assim como as pessoas que fazem o brainstorming para um slogan publicitário sabem algo sobre o produto antes do início da sessão, o brainstorming para aulas exige que os participantes saibam algo sobre o problema. Cada membro da equipe deve entrar na sessão de brainstorming com uma compreensão clara do problema a ser resolvido. Sem dúvida, durante a preparação, cada membro da equipe terá gerado sua própria lista preliminar de classes.

Embora o brainstorming geralmente seja uma atividade em grupo, você pode praticá-lo sozinho em problemas menores.

Filtragem (Filtering)

O brainstorming produz uma lista provisória de classes. A próxima fase é pegar essa lista e determinar quais são as classes principais na solução do problema. Talvez duas classes na lista sejam, na verdade, a mesma coisa. Essas classes duplicadas geralmente surgem porque pessoas em diferentes partes de uma organização usam nomes diferentes para o mesmo conceito ou entidade. Além disso, duas classes na lista podem ter muitos atributos e comportamentos comuns que podem ser combinados.

Algumas classes podem não pertencer à solução do problema. Por exemplo, se estivermos simulando uma calculadora, podemos listar o usuário como uma classe possível. Na realidade, o usuário não faz parte do funcionamento interno da simulação como uma classe; o usuário é uma entidade externa ao problema que fornece entrada para a simulação. Outra classe possível pode ser o botão **ligar**. Uma breve reflexão, no entanto, mostra que o botão ligar não faz parte da simulação; em vez disso, é o que inicia a execução do programa de simulação.

Conforme a filtragem é concluída, a lista de classes remanescente é passada para a próxima etapa.

Cenários (Scenarios)

O objetivo da fase de cenários é atribuir responsabilidades a cada classe. As responsabilidades são eventualmente implementadas como subprogramas. Nesta etapa, estamos interessados apenas em **quais** são as tarefas, não em como elas podem ser executadas.

Existem dois tipos de responsabilidades: o que uma classe deve saber sobre si mesma (conhecimento) e o que uma classe deve ser capaz de fazer (comportamento). Uma classe **encapsula** seus dados (conhecimento), de tal forma que objetos em uma classe não podem acessar diretamente dados em outra classe. O encapsulamento é o agrupamento de dados e ações de forma que as propriedades lógicas dos dados e ações sejam separadas dos detalhes de implementação. O encapsulamento é a chave para a abstração. Ao mesmo tempo, cada classe tem a responsabilidade de disponibilizar dados (conhecimento) para outras classes que precisam deles. Portanto, cada classe tem a responsabilidade de saber as coisas sobre si mesma que as outras precisam obter. Por exemplo, uma classe de aluno deve "saber" seu nome e endereço, e uma classe que usa a classe de aluno deve ser capaz de "obter" essas informações. Essas responsabilidades geralmente são nomeadas com "Get" precedendo o nome dos dados – por exemplo, GetName ou GetEmailAddress. Se o endereço de e-mail é mantido na classe de aluno ou se a classe de aluno deve solicitar que outra classe acesse o endereço é irrelevante nesta fase: o fato importante é que a classe de aluno conhece seu próprio endereço de e-mail e pode devolvê-lo a uma classe que precise dele.

* Encapsulamento: Agrupamento de dados e ações para que as propriedades lógicas de dados e ações sejam separadas dos detalhes de implementação.

As responsabilidades pelo comportamento se parecem mais com as tarefas que descrevemos no design de cima para baixo. Por exemplo, uma responsabilidade pode ser a de uma turma de alunos calcular sua média das notas. Em um projeto top-down, diríamos que uma tarefa é calcular o dado média das notas. Em um projeto orientado a objetos, dizemos que a turma de alunos é responsável por calcular sua própria média das notas. A distinção é sutil e profunda. O código final para o cálculo pode parecer o mesmo, mas é executado de maneiras diferentes. Em um programa baseado em um design top-down, o programa chama o subprograma que calcula a média das notas, passando o objeto aluno como parâmetro. Em um programa orientado a objetos, uma mensagem é enviada ao objeto da turma para calcular sua média de notas. Não há parâmetros porque o objeto para o qual a mensagem é enviada conhece seus próprios dados.

O nome desta fase dá uma pista sobre como atribuir responsabilidades às turmas. A equipe (ou um indivíduo) descreve diferentes cenários de processamento envolvendo as turmas. Os cenários são roteiros "**e se**" que permitem aos participantes representar diferentes situações ou a um indivíduo pensar sobre elas.

O resultado desta fase é um conjunto de turmas com as responsabilidades de cada turma atribuídas, talvez escritas em um cartão CRC. As responsabilidades de cada turma são listadas no cartão, juntamente com as turmas com as quais uma responsabilidade deve colaborar.

Responsabilidades de Algoritmo (Responsibility)

Finalmente, algoritmos devem ser escritos para as responsabilidades. Como o processo de resolução de problemas se concentra em dados e não em ações na visão de design orientada a objetos, os algoritmos para executar responsabilidades tendem a ser bastante curtos. Por exemplo, as responsabilidades de conhecimento geralmente apenas retornam o conteúdo de uma das variáveis de um objeto ou enviam uma mensagem para outro objeto para recuperá-lo. As responsabilidades de ação são um pouco mais complicadas, frequentemente envolvendo cálculos. Portanto, o método top-down de projetar um algoritmo geralmente é apropriado para projetar algoritmos de responsabilidade de ação.

Considerações Finais

Para resumir, os métodos de projeto top-down focam no **processo** de transformar a entrada em saída, resultando em uma hierarquia de tarefas. O projeto orientado a objetos foca nos **objetos de dados** que devem ser transformados, resultando em uma hierarquia de objetos. Grady Booch coloca desta forma: "Leia a especificação do software que você deseja construir. Sublinhe os verbos se você busca código procedural, os substantivos se você almeja um programa orientado a objetos."[2]

Propomos que você circule os substantivos e sublinhe os verbos como forma de começar. Os substantivos se tornam objetos; os verbos se tornam operações. Em um projeto top-down, os verbos são o foco principal; Em um design orientado a objetos, os substantivos são o foco principal.

Agora, vamos analisar um exemplo.

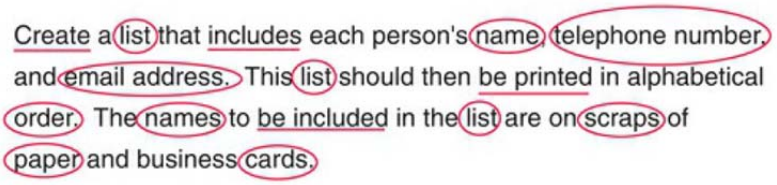
Exemplo

O Problema

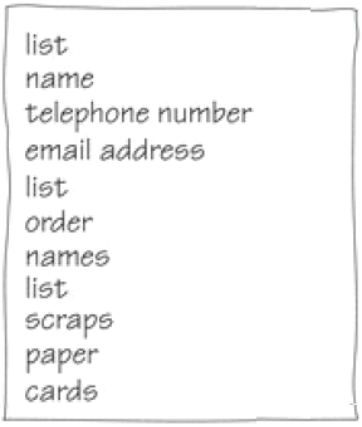
Crie uma lista que inclua o nome, o número de telefone e o endereço de e-mail de cada pessoa. Esta lista deve ser impressa em ordem alfabética. Os nomes a serem incluídos na lista estão em pedaços de papel e cartões de visita.

Brainstorming e Filtragem

Na figura a seguir, vamos tentar circular os substantivos e sublinhar os verbos.

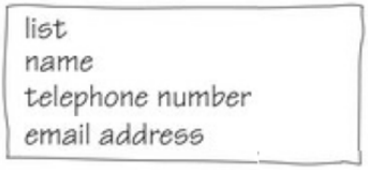


A primeira passagem por uma lista de classes incluiria conforme o descrito ao lado.



Três dessas classes são iguais: As três referências a "**list**" referem-se ao contêiner que está sendo criado. "**Order**" é um substantivo, mas o que é uma **classe de ordem**? Na verdade, ela descreve como a classe deve imprimir seus itens. Portanto, a descartamos como uma classe. "**Name**" e "**names**" devem ser combinados em uma classe.

"**Scraps**", "**paper**" e "**cards**" descrevem objetos que contêm os dados no mundo real. Eles não têm contrapartida no design. Nossa lista filtrada é mostrada ao lado.



Os verbos na declaração do problema nos dão uma vantagem sobre as responsabilidades: "create", "print" e "include". Assim como "scraps", "paper" e "cards", "include" é uma instrução para alguém que está

preparando os dados e não tem contrapartida no design. No entanto, indica que devemos ter um objeto que insira os dados a serem colocados na lista. Exatamente o que são esses dados? São o nome, o número de telefone e o endereço de e-mail de cada pessoa na lista. Mas essa linha de pensamento leva à descoberta de que perdemos uma pista importante na definição do problema. Um adjetivo possessivo, "pessoa", na verdade nomeia uma classe importante; **nome**, **número de telefone** e **endereço de e-mail** são classes que ajudam a definir (estão contidas em) uma classe de **pessoa**.

Agora temos uma escolha de design. A classe **pessoa** deve ter a responsabilidade de inserir seus próprios dados para se inicializar, ou devemos criar outra classe que faça a entrada e envie os dados para inicializar a classe **pessoa**? Vamos fazer com que a classe **pessoa** seja responsável por se inicializar. A classe **pessoa** também deve ser responsável por imprimir a si mesma.

A classe **pessoa** colabora com alguma outra classe? A resposta a esta pergunta depende de como decidimos representar os dados na classe **pessoa**. Representamos **nome**, **número de telefone** e **endereço de e-mail** como itens de dados simples dentro da classe **pessoa**, ou representamos cada um como sua própria classe? Vamos representar **nome** como uma classe com dois itens de dados, **nome** e **sobrenome**, e fazer com que os outros sejam variáveis de string na classe **pessoa**. Ambas as classes **pessoa** e **nome** devem ter responsabilidades de conhecimento sobre seus valores de dados. Aqui estão os cartões CRC para essas classes.

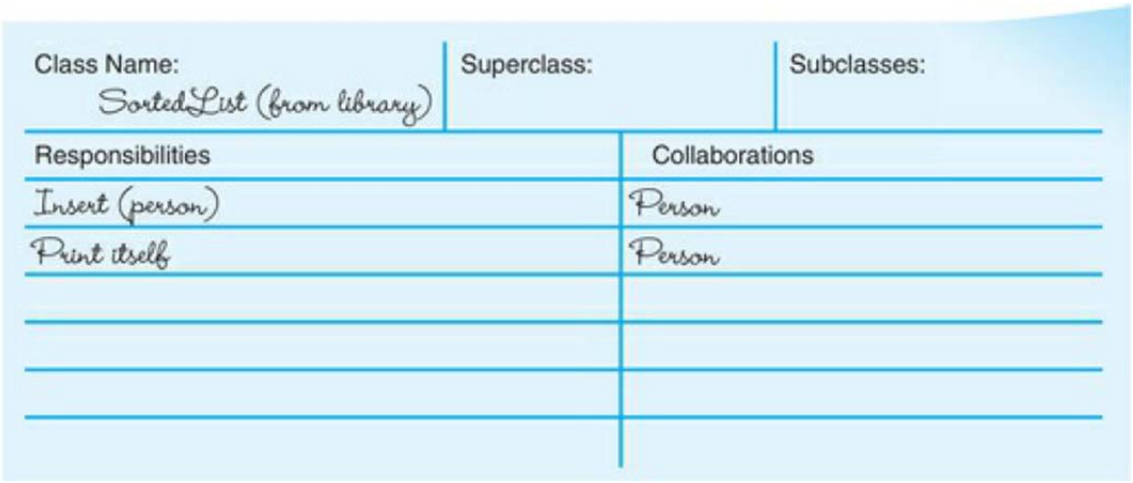
Class Name: Person	Superclass:	Subclasses:
Responsibilities	Collaborations	
Initialize itself (name, telephone, email)	Name, String	
Print	Name, String	
GetEmail	String	
GetName	Name, String	
GetTelephone	String	

Class Name: Name	Superclass:	Subclasses:
Responsibilities	Collaborations	
Initialize itself (firstName, lastName)	String	
Print itself	String	
GetFirstName	String	
GetLastName	String	

E quanto ao objeto lista? A lista deve manter os itens em ordem alfabética ou deve classificá-los antes de imprimi-los? Cada linguagem na qual podemos implementar este projeto possui uma biblioteca de classes de contêiner disponíveis para uso. Vamos usar uma dessas classes, que mantém a lista em ordem alfabética. Esta classe de biblioteca também deve imprimir a lista. Podemos criar um cartão CRC para esta classe, mas observe que ela provavelmente será implementada usando uma classe de biblioteca (library class).

? Cuidado com software pirata

Um estudo de 2013 analisou 270 sites e redes peer-to-peer, 108 downloads de software, 155 CDs ou DVDs, 2077 entrevistas com consumidores e 258 entrevistas com gerentes de TI em todo o mundo. Os resultados mostraram que, dos softwares falsificados que não acompanham o computador, 45% são baixados da internet. Dos softwares baixados de sites ou redes peer-to-peer, 78% incluem algum tipo de spyware e 36% contêm trojans e adware.[3]



Por convenção, quando uma classe atinge o estágio CRC, iniciamos seu identificador com uma letra maiúscula.

Responsabilidade (Responsibility)

A classe **Pessoa**. Há duas responsabilidades a serem decompostas: inicializar e imprimir. Como **Nome** é uma classe, podemos simplesmente deixá-la inicializar e imprimir a si mesma. Aplicamos um subprograma (método) a um objeto colocando o nome do objeto antes do nome do método com um ponto entre.

```
Initialize
   name.initialize()
   write "Entre com o número de telefone: "
   Get telephoneNumber
   write "Entre com o endereço de e-mail: "
   Get emailAddress
```

```
Print
   name.print()
   write "Telephone number: " + telephoneNumber
   write "E-mail address: " + emailAddress
```

A classe **Nome**. Esta classe tem as mesmas duas responsabilidades: inicializar e imprimir. No entanto, os algoritmos são diferentes. Para a responsabilidade de inicialização, o usuário deve ser solicitado a inserir o nome e o algoritmo deve ler o nome. Para a responsabilidade de impressão, o primeiro e o último nome devem ser exibidos com os rótulos apropriados.

```
Initialize
   write "Entre com o primeiro nome: "
   read firstName
   write "Entre com o ultimo nome: "
   read lastName
```

```
Print
   print "Fisrt name: ", firstName
   print "Last name: ", lastName
```

Paramos o projeto neste ponto. Releia o início do Capítulo 7, onde discutimos a resolução de problemas e o processo de projeto top-down. Um projeto top-down produz uma árvore hierárquica com tarefas nos nós da árvore. O projeto orientado a objetos produz um conjunto de classes, cada uma com responsabilidades por seu próprio comportamento. Uma é melhor que a outra? Bem, o projeto orientado a objetos cria classes que podem ser úteis em outros contextos. A reusabilidade é uma das grandes vantagens de um projeto orientado a objetos. Classes projetadas para um problema podem ser usadas em outro problema, porque cada classe é autocontida; ou seja, cada classe é responsável por seu próprio comportamento.

Você pode pensar na fase de resolução de problemas orientados a objetos como o mapeamento dos objetos do mundo real em classes, que são descrições das categorias de objetos. A fase de implementação pega as descrições das categorias (classes) e cria instâncias das classes que simulam os objetos no

problema. As interações dos objetos no programa simulam a interação dos objetos no mundo real do problema. A FIGURA 9.1 resume esse processo.

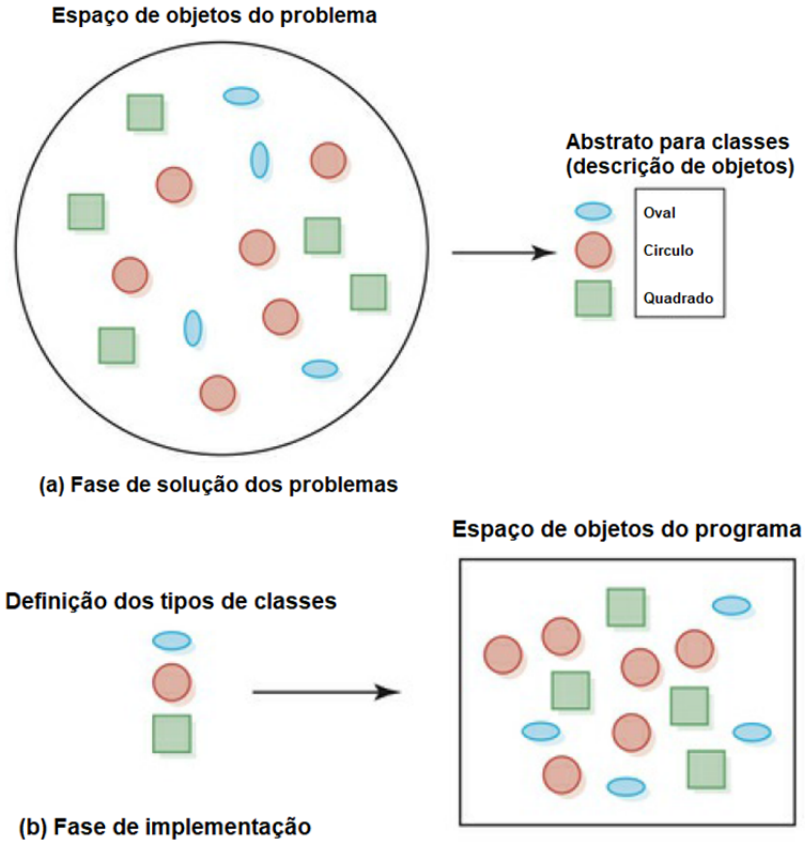


FIGURA 9.1 Mapeamento de um problema em uma solução.

9.2 Processo de Tradução

Lembre-se do Capítulo 6 que um programa escrito em linguagem assembly é inserido no **assembler**, que traduz as instruções em linguagem assembly para código de máquina. O código de máquina, que é a saída do assembler, é então executado. Com linguagens de alto nível, empregamos outras ferramentas de software para auxiliar no processo de tradução. Vejamos a função básica dessas ferramentas antes de examinar as linguagens de alto nível.

Compiladores

Os algoritmos que traduzem instruções em linguagem assembly para código de máquina são muito simples porque as linguagens assembly são muito simples. Por "simples", queremos dizer que cada instrução realiza uma operação fundamental. Linguagens de alto nível fornecem um conjunto mais rico de instruções que tornam a vida do programador ainda mais fácil, mas como as construções são mais abstratas, o processo de tradução é mais difícil. Programas que traduzem programas escritos em uma linguagem de alto nível são chamados de compiladores. Nos primórdios da programação de computadores, a saída de um compilador era uma versão em linguagem assembly do programa, que precisava ser executada por um montador para finalmente executar o programa em linguagem de máquina. À medida que os cientistas da computação passaram a compreender melhor o processo de tradução, os compiladores se tornaram mais sofisticados e a fase de linguagem assembly foi frequentemente eliminada. Veja a FIGURA 9.2.



FIGURA 9.2 Processo de compilação.

Um programa escrito em uma linguagem de alto nível pode ser executado em qualquer computador que tenha um compilador apropriado para a linguagem. Um compilador é um programa; portanto, uma versão em código de máquina do compilador deve estar disponível para que uma máquina específica possa compilar um programa. Assim, para ser usado em vários tipos de máquinas, cada linguagem de alto nível deve ter muitos compiladores para essa linguagem.

* Compilador: Um programa que traduz um programa de linguagem de alto nível para código de máquina.

Interpretadores

Um interpretador é um programa que traduz e executa as instruções em sequência. Ao contrário de um montador ou compilador que produz código de máquina como saída, que é então executado em uma etapa separada, um interpretador traduz uma instrução e a executa imediatamente. Interpretadores podem ser vistos como **simuladores** ou **máquinas virtuais** que entendem a linguagem em que um programa é escrito. Como Terry Pratt aponta em seu texto clássico sobre linguagens de programação, tanto um tradutor quanto um simulador aceitam programas em uma linguagem de alto nível como entrada. O tradutor (montador ou compilador) simplesmente produz um programa equivalente na linguagem de máquina apropriada, que deve então ser executado. O simulador executa o programa de entrada diretamente.[4]

* Interpretador: Um programa que insere um programa em uma linguagem de alto nível e instrui o computador a executar as ações especificadas em cada instrução.

As linguagens de programação de alto nível de segunda geração se apresentaram em duas variedades: as compiladas e as interpretadas. As linguagens Fortran, Cobol e Algol foram compiladas; As linguagens Lisp, Snobol4 e APL foram interpretadas. Devido à complexidade dos interpretadores de software, os programas em linguagens interpretadas geralmente eram executados muito mais lentamente do que os programas compilados. Como resultado, a tendência foi em direção às linguagens compiladas – até o advento do Java.

O Java foi introduzido em 1996 e conquistou a comunidade da computação. No projeto do Java, a portabilidade era de suma importância. Para atingir a portabilidade ideal, o Java é compilado em uma linguagem de máquina padrão chamada Bytecode. Mas como pode haver uma **linguagem de máquina padrão**? Um interpretador de software chamado JVM (Java Virtual Machine) pega o programa Bytecode e o executa. Ou seja, o Bytecode não é a linguagem de máquina para nenhum processador de hardware específico. Qualquer máquina que tenha uma JVM pode executar o programa Java compilado.

* Bytecode: Uma linguagem de máquina padrão na qual o código-fonte Java é compilado.

A portabilidade alcançada por linguagens de alto nível padronizadas não é a mesma que a portabilidade alcançada pela tradução de Java para Bytecode e sua posterior interpretação em uma JVM. Um programa escrito em uma linguagem de alto nível pode ser compilado e executado em qualquer máquina que tenha o compilador apropriado; o programa é traduzido em código de máquina que é diretamente executado por um computador. Um programa Java é compilado em Bytecode, e o programa compilado em Bytecode pode ser executado em qualquer máquina que tenha um interpretador JVM. Ou seja, a saída do compilador Java é interpretada, não executada diretamente. (Vide a FIGURA 9.3) Java é sempre traduzido para Bytecode. Além disso, existem compiladores para outras linguagens que traduzem a linguagem para Bytecode em vez de código de máquina. Por exemplo, existem versões de compiladores Ada que traduzem Ada para Bytecode.

? O sistema p-system da UCSD é anterior ao Bytecode

Na década de 1970, a Universidade da Califórnia em San Diego tinha um sistema que executava p-code (código p), uma linguagem muito semelhante ao Bytecode. Programas escritos em Pascal e Fortran eram traduzidos para p-code, que podia ser executado em qualquer hardware com um interpretador de p-code.

A JVM é uma máquina virtual, assim como o sistema Pep/8 discutido no Capítulo 6. Nesse capítulo, definimos uma máquina virtual como uma máquina hipotética projetada para ilustrar recursos importantes de uma máquina real. A JVM é uma máquina hipotética projetada para executar Bytecode.

9.3 Paradigmas de Linguagens de Programação

O que é um **paradigma**? O American Heritage Dictionary of the English Language (ou Dicionário Americano Heritage da Língua Inglesa) fornece duas definições relacionadas a como nós, na computação, usamos o termo: **"**aquele que serve como um padrão ou modelo**"** e **"**um conjunto de suposições, conceitos, valores e

práticas que constituem uma maneira de visualizar a realidade para a comunidade que os compartilha, especialmente em uma disciplina intelectual" [5]. No Capítulo 1, descrevemos a história do desenvolvimento de software, listando algumas das linguagens de programação que foram desenvolvidas em cada geração. Outra maneira de visualizar as linguagens de programação é observar como diferentes linguagens refletem diferentes visões da realidade – isto é, observar os diferentes paradigmas representados.

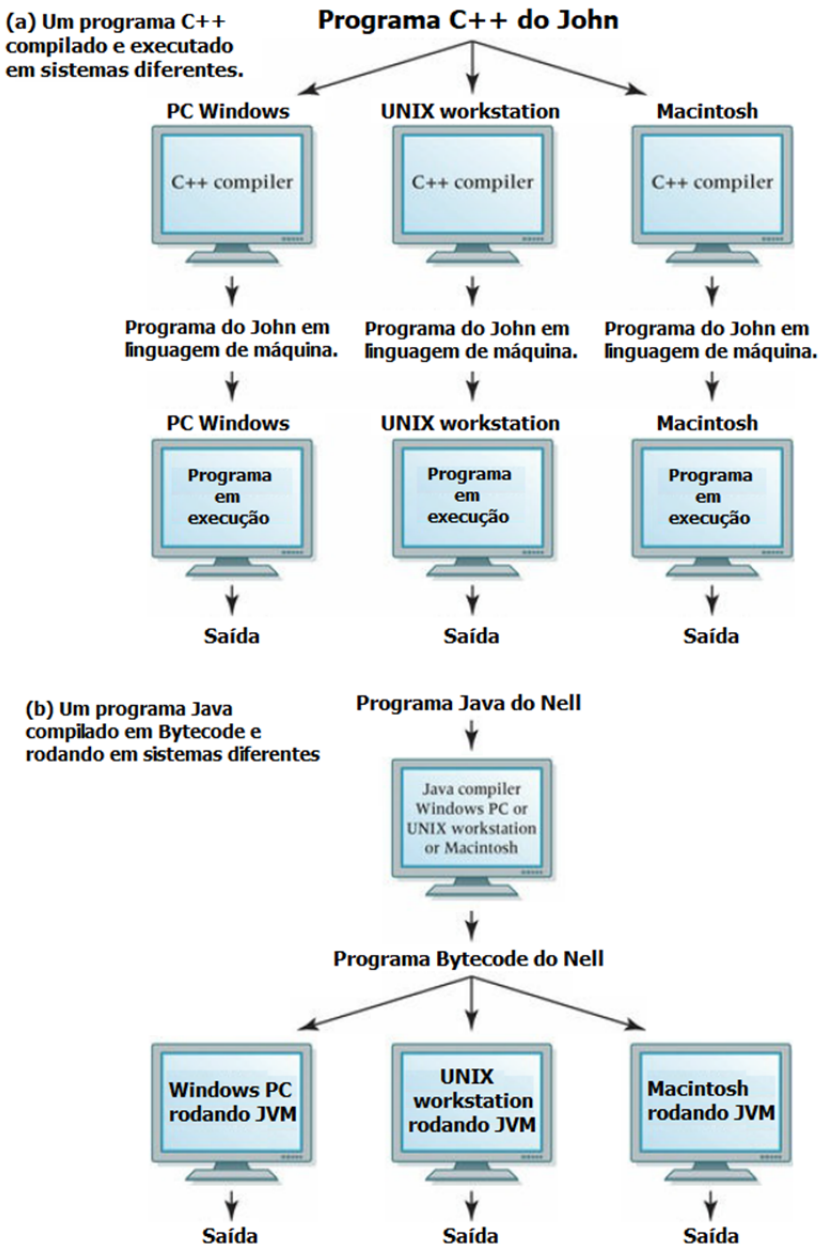


FIGURA 9.3 Portabilidade proporcionada por linguagens padronizadas versus interpretação por Bytecode.

Existem dois paradigmas principais, **imperativo** e **declarativo**, e muitos subparadigmas dentro de cada um. Posteriormente, examinaremos as diferentes linguagens dentro desses paradigmas.

Paradigma Imperativo

O modelo de von Neumann de instruções sequenciais que operam sobre valores na memória influenciou significativamente o modelo mais comum de linguagem de programação: o modelo imperativo.

As linguagens dominantes usadas na indústria ao longo da história do software de computação vêm desse paradigma. Essas linguagens incluem Fortran, Basic, C, Pascal e C++. Nesse paradigma, o programa descreve o processamento necessário para resolver o problema. O paradigma imperativo é, portanto, caracterizado pela execução sequencial de instruções, o uso de variáveis que representam posições de memória e o uso de instruções de atribuição que alteram os valores dessas variáveis.[7]

? O que é um paradigma?

Uma busca na internet resulta em muitas definições relevantes, incluindo "um padrão ou um exemplo de algo". A palavra também conota as ideias de uma imagem mental e um padrão de pensamento. Thomas Kuhn usa a palavra para significar o modelo que os cientistas têm sobre uma área específica do conhecimento. O famoso livro de Kuhn, The Structure of Scientific Revolutions (ou A Estrutura das Revoluções Científicas), apresenta sua visão dos estágios pelos quais uma ciência passa para passar de um paradigma a outro.[6]

Paradigma Procedural

A **programação procedural** é um modelo imperativo no qual as instruções são agrupadas em subprogramas. Um programa é uma hierarquia de subprogramas, cada um dos quais executa uma tarefa específica necessária à solução do programa geral. Nossos exemplos de pseudocódigo seguem esse modelo. Escrevemos subprogramas e passamos a eles os dados necessários para realizar sua função.

Paradigma Orientado a Objetos

A visão orientada a objetos é a de um mundo de objetos interagindo. Cada objeto é responsável por suas próprias ações. No paradigma procedural, os dados são considerados passivos e o programa age sobre eles. No paradigma orientado a objetos, os objetos de dados são ativos. Os objetos e o código que os manipula são agrupados, tornando cada objeto responsável por sua própria manipulação. A Simula e a Smalltalk foram as duas primeiras linguagens de programação orientadas a objetos. Java e Python são duas linguagens modernas orientadas a objetos.

C++ e Java são linguagens imperativas que são um tanto quanto mistas em termos de seu paradigma. Embora Java seja considerada orientada a objetos, ela possui alguns recursos procedurais. C++ é considerada procedural, mas possui alguns recursos orientados a objetos.

Exemplos de linguagens específicas são apresentados neste livro, mas não pretendem ser abrangentes o suficiente para fornecer ao aluno um conhecimento prático da linguagem. No entanto, os suplementos disponíveis para este livro incluem exercícios de laboratório para diversas linguagens, incluindo Java e C++, que se aprofundam mais.

Paradigma Declarativo

O **paradigma declarativo** é um modelo no qual os resultados são descritos, mas as etapas para obtê-los não são declaradas. Existem dois modelos básicos dentro deste paradigma: **funcional** e **lógico**.

Modelo Funcional

O **modelo funcional** é baseado no conceito matemático de função. A computação é expressa em termos da avaliação de funções; a solução para um problema é expressa em termos de chamadas de função. Portanto, o mecanismo básico é a avaliação de funções, e não há variáveis nem instruções de atribuição. Por exemplo, a adição de dois valores seria expressa desta forma:

```
(+ 30 40)
```

onde os parênteses representam uma expressão a ser avaliada aplicando o primeiro item (que deve ser uma função) ao restante da lista. Esta expressão é avaliada aplicando a função de adição aos próximos dois números, que retorna o valor 70. Não há construção de loop; a repetição é expressa em termos de chamadas de função recursivas. As linguagens mais conhecidas no paradigma funcional são Lisp, Scheme (um derivado de Lisp) e ML.

Vamos examinar uma série de expressões Scheme que ilustram a essência da linguagem. Scheme é interpretado, então o resultado segue imediatamente a instrução. Um interpretador usa **#;>** como um prompt para inserir uma expressão; usaremos isso aqui.[8] As linhas sem o prompt são as respostas do sistema (o que o sistema retorna).

```
#;>  (* 3 4)
12
#;>  (+ (* 5 4)(+ 1 4))
25
#;>  (length '(2 4 6 8 10))
5
#;>  (max 2 5 1 3)
5
```

Na primeira expressão, 3 é multiplicado por 4, resultando em 12. Na segunda expressão, os resultados da multiplicação de 5 por 4 e da adição de 1 e 4 são somados, resultando em 25. A terceira expressão solicita o número de itens na lista seguinte à lista indicada pelo sinal '. Na quarta expressão, o máximo dos seguintes valores é retornado.

No Capítulo 7, escrevemos um algoritmo recursivo para calcular o fatorial de um número. **lambda** é a palavra para definir uma função. Aqui (ao lado) está o código do Scheme correspondente – compare-o com o primeiro algoritmo.

Após a definição da função **factorial**, fornecer o nome e o argumento entre parênteses executa a função, retornando o valor 5040.

```
#;>  (define factorial
#;>  (lambda (n)
#;>     (if
#;>     (= n 0)
#;>  1
#;>  (* n (factorial (- n 1))))))
#;>  (factorial 7)
5040
```

Programação Lógica

A **programação lógica** é baseada nos princípios da lógica simbólica. Este modelo compreende um conjunto de fatos sobre objetos e um conjunto de regras sobre os relacionamentos entre os objetos. Um programa consiste em fazer perguntas sobre esses objetos e suas relações, que podem ser deduzidas a partir dos fatos e das regras. O algoritmo de resolução de problemas subjacente usa as regras da lógica para deduzir a resposta a partir dos fatos e das regras.

Prolog é uma linguagem de programação lógica de terceira geração desenvolvida na França em 1970. Ganhou destaque em 1981, quando pesquisadores japoneses anunciaram que a programação lógica desempenharia um papel importante em seus computadores de quinta geração. Um programa Prolog consiste em três tipos de instruções: um tipo declara fatos sobre objetos e seus relacionamentos entre si; outro tipo define regras sobre objetos e seus relacionamentos; e um terceiro tipo faz perguntas sobre os objetos e seus relacionamentos.[9]

Por exemplo, o código a seguir define um conjunto de fatos que relacionam animais de estimação a seus donos.

```
owns(mary, bo).
owns(ann, kitty).
owns(bob, riley).
owns(susy, charlie).
```

Aqui, **owns** é o nome do relacionamento, os objetos estão entre parênteses e o ponto final encerra a declaração do fato. Isso significa que **mary** possui **bo** ou **bo** possui **mary**? Isso fica a critério do programador. Ele ou ela deve ser consistente em sua interpretação.

Quando você tem um banco de dados de fatos, o Prolog permite que você faça perguntas sobre o banco de dados. Observe as três instruções do Prolog a seguir.

```
?-owns(mary, bo).
?-owns(bo, mary).
?-owns(susy, bo).
```

O sistema Prolog responde **yes** à primeira, **no** à segunda e **no** à terceira.

Em Prolog, uma constante começa com uma letra minúscula e uma variável começa com uma letra maiúscula. Fazemos perguntas sobre fatos substituindo uma variável por uma constante em um fato.

```
?-owns(ann, Cat).
?-owns(Name, charlie).
```

Neste exemplo, a primeira instrução retorna **Cat = kitty**. A segunda retorna **Name = susy**.

Tanto Lisp quanto Prolog são usados em aplicações de inteligência artificial (descritas no Capítulo 13). Como você pode ver, os programas nessas linguagens têm pouca semelhança com a arquitetura de von Neumann refletida em linguagens no paradigma imperativo, conforme representado em nosso pseudocódigo.

9.4 Funcionalidade em Linguagens de Alto Nível

Duas construções de pseudocódigo – seleção e repetição (looping) – são características das linguagens imperativas. No Capítulo 6, implementamos essas construções em linguagem assembly, mostrando o quão detalhadas as instruções precisavam ser. Também examinamos essas construções juntamente com

subprogramas em pseudocódigo. Em linguagens de alto nível, seleção e repetição são muito fáceis. Subprogramas e passagem de parâmetros, no entanto, são mais complicados.

Primeiro, revisamos o conceito de expressão booleana, que é a construção que as linguagens de alto nível usam para fazer escolhas. Em seguida, examinamos outras construções que as linguagens de alto nível fornecem para tornar a programação mais fácil e segura.

Expressões Booleanas

No Capítulo 6, escrevemos um algoritmo para ler pares de números e imprimi-los em ordem. Ele continha uma instrução de seleção dentro de um laço. Aqui está o esboço do laço com a instrução de seleção.

```
...
while (numberRead < numberOfPairs)
   ...
   if (number1 < number2)
      print number1, " ", number2
   else
      print number2, " ", number1
```

Cada uma dessas declarações faz uma pergunta. Observe como essas perguntas são formuladas:

```
(numberRead < numberOfPairs)
(number1 < number2)
```

Cada frase é, na verdade, uma declaração. Se a declaração for verdadeira, a resposta à pergunta é verdadeira. Se a declaração não for verdadeira, a resposta à pergunta é falsa. Fazer declarações e então testar se elas são verdadeiras ou falsas é como as linguagens de programação fazem perguntas. Essas declarações são chamadas de **asserções** ou **condições**. Quando escrevemos algoritmos, fazemos asserções em declarações semelhantes ao inglês. Quando os algoritmos são traduzidos para uma linguagem de programação de alto nível, as declarações semelhantes ao inglês são reescritas como expressões booleanas.

O que é uma expressão booleana? No Capítulo 4, introduzimos o conceito de operações booleanas quando discutimos portas e circuitos. Aqui, as usamos no nível lógico em vez do nível de hardware. Uma expressão booleana é uma sequência de identificadores, separados por operadores compatíveis, que avaliam como verdadeiro ou falso. Uma expressão booleana pode ser qualquer uma das seguintes.

- Uma variável booleana.
- Uma expressão aritmética seguida por um operador relacional seguido por uma expressão aritmética.
- Uma expressão booleana seguida por um operador booleano seguido por uma expressão booleana.

* Expressão booleana: Uma sequência de identificadores, separados por operadores compatíveis, que pode ser avaliada como verdadeiro ou falso.

Até agora, em nossos exemplos, as variáveis continham valores numéricos. Uma variável booleana é um local na memória referenciado por um identificador que pode conter TRUE (verdadeiro) ou FALSE (falso). (Ao nos referirmos ao código em uma linguagem específica ou ao que está realmente armazenado na memória, usamos uma fonte monoespaçada, ou de código.)

Um operador relacional é aquele que compara dois valores. Os seis operadores relacionais estão resumidos no gráfico a seguir, juntamente com os símbolos que várias linguagens de alto nível usam para representar a relação.

Um operador relacional entre duas expressões aritméticas pergunta se a relação existe entre as duas expressões. Por exemplo,

```
xValue < yValue
```

está fazendo a afirmação de que **xValue** é menor que **yValue**. Se **xValue** for menor que **yValue**, então o resultado da expressão é TRUE; se **xValue** não for menor que **yValue**, então o resultado é FALSE.

Para evitar confusão sobre o uso de = e == para significar igualdade em diferentes linguagens de programação, usamos a palavra "igual" em vez de escolher um dos símbolos em nossos algoritmos. Lembre-se de que os três operadores booleanos são os operadores especiais AND, OR e NOT. O operador AND retorna TRUE se ambas as expressões forem TRUE e FALSE caso contrário. O operador OR retorna FALSE se ambas as expressões forem FALSE e TRUE caso contrário. O operador NOT altera o valor da expressão.

Símbolo	Significado	Exemplo	Evolução
<	Menor que	`number1 < number2`	TRUE se number1 é menor que number2, do contrário, FALSE.
<=	Menor que ou igual	`number1 <= number2`	TRUE se number1 é menor que ou igual a number2, do contrário, FALSE.
>	Maior que	`number1 > number2`	TRUE se number1 é maior que number2, do contrário, FALSE.
>=	Maior que ou igual	`number1 >= number2`	TRUE se number1 é maior que ou igual a number2, do contrário, FALSE.
!= ou <> ou /=	Não igual (ou diferença)	`number1 != number2`	TRUE se number1 é não igual a number2, do contrário, FALSE.
= ou ==	Igual	`number1 == number2`	TRUE se number1 é igual a number2, do contrário, FALSE.

Tipagem de Dados

Ao trabalhar em uma linguagem assembly, atribuímos identificadores a locais de memória sem considerar o que deve ser armazenado nesses locais. Muitas linguagens de alto nível amplamente utilizadas (incluindo C++, Java e PHP) exigem que você indique o que pode ser armazenado em um local ao associá-lo a um identificador. Se uma instrução em um programa tentar armazenar um valor em uma variável que não seja do tipo apropriado, uma mensagem de erro será emitida. O requisito de que apenas um valor do tipo apropriado possa ser armazenado em uma variável é chamado de **tipagem forte**.

* Tipagem forte: cada variável recebe um tipo, e somente valores desse tipo podem ser armazenados na variável.

Por exemplo, observe os oito bits a seguir: 00110001. O que eles representam? É um byte na memória. Sim, mas o que significa? Bem, pode ser a representação binária do número decimal 49. Também pode ser a representação ASCII estendida do caractere '1'. Poderia significar alguma outra coisa? Sim, poderia ser o especificador de instruções Pep/8 para a instrução de captura de modo direto DCI. Portanto, quando um programa está em execução, ele deve saber como interpretar o conteúdo de um local na memória.

Nas próximas seções, examinaremos os tipos comuns de valores de dados e exploraremos como as linguagens de alto nível permitem associar localizações a identificadores. Cada um desses tipos de dados possui certas operações que podem ser aplicadas legalmente aos valores do tipo. Um tipo de dado é uma descrição do conjunto de valores e do conjunto básico de operações que podem ser aplicadas aos valores do tipo.

Das linguagens que exploramos, C++, Java e VB .NET são fortemente tipadas; Python não é.

* Tipo de dado: Uma descrição do conjunto de valores e do conjunto básico de operações que podem ser aplicadas aos valores do tipo.

Tipos de Dados

Dados são os símbolos físicos que representam informações. Dentro de um computador, tanto dados quanto instruções são apenas padrões de bits binários. O computador executa uma instrução porque o endereço da instrução é carregado no contador de programa (PC) e a instrução é então carregada no registrador de instruções (RI). Esse mesmo padrão de bits executado também pode representar um número inteiro, um número real, um caractere ou um valor booleano. O ponto principal é que o computador interpreta o padrão de bits como o que ele espera que seja.

? Considere a palavra "bow" (arco).

Uma palavra é uma sequência de símbolos retirados do alfabeto. Algumas sequências ou padrões de símbolos receberam significados; outros não. "Bordado" é uma palavra em inglês. No entanto, pode significar coisas diferentes: parte de um navio, algo que uma menina usa no cabelo, algo com que se toca violino ou o ato de se curvar a partir da cintura. Podemos diferenciar os significados com base no contexto da palavra, assim como um compilador pode diferenciar com base na sintaxe ao redor.

Por exemplo, no Pep/8, a instrução para Parar (stop) é um byte com todos os bits zero. Quando esta instrução é carregada no registrador de instruções, o programa para. Um byte com todos os bits zero também pode ser interpretado como um número binário de oito bits contendo o valor 0 (zero). Se a

localização contendo todos os bits zero for adicionada ao conteúdo de um registrador, o valor é interpretado como um número.

A maioria das linguagens de alto nível possui quatro tipos de dados distintos incorporados: números inteiros, números reais, caracteres e valores booleanos.

Inteiros: O tipo de dado inteiro representa um intervalo de valores inteiros, do menor ao maior. O intervalo varia dependendo de quantos bytes são atribuídos para representar um valor inteiro. Algumas linguagens de alto nível fornecem vários tipos inteiros de tamanhos diferentes, o que permite ao usuário escolher aquele que melhor se adapta aos dados em um problema específico.

As operações que podem ser aplicadas a números inteiros são os operadores aritméticos e relacionais padrão. Adição e subtração são representadas pelos símbolos padrão + e -. Multiplicação e divisão são geralmente representadas por * e /. Dependendo da linguagem, a divisão inteira pode retornar um número real ou o quociente inteiro. Algumas linguagens têm dois símbolos para divisão: um que retorna um resultado real e outro que retorna o quociente inteiro. A maioria das linguagens também possui um operador que retorna o resto inteiro da divisão. Esse operador é chamado de **operador de módulo**, mas pode ou não atuar como o operador de módulo matemático. Os operadores relacionais são representados pelos símbolos mostrados na tabela da seção anterior.

Reais: O tipo de dado real também representa um intervalo do menor ao maior valor com uma precisão dada. Assim como o tipo de dado inteiro, o intervalo varia dependendo do número de bytes atribuídos para representar um número real. Muitas linguagens de alto nível têm dois tamanhos de números reais. As operações que podem ser aplicadas a números reais são as mesmas que podem ser aplicadas a números inteiros. No entanto, é preciso ter cuidado ao aplicar os operadores relacionais a valores reais, pois números reais geralmente não são exatos. Por exemplo, 1/3 + 1/3 + 1/3 em aritmética computacional não é necessariamente 1,0. Na verdade, 1/10 * 10 não é 1,0 em aritmética computacional.

Caracteres: No Capítulo 3, dissemos que um mapeamento do conjunto de caracteres ASCII para código requer apenas um byte. Um mapeamento comumente usado para o conjunto de caracteres Unicode usa dois bytes. Nesse mapeamento, nosso alfabeto inglês é representado em ASCII, que é um subconjunto do Unicode. Aplicar operações aritméticas a caracteres não faz muito sentido, e muitas linguagens fortemente tipadas não permitem isso. No entanto, comparar caracteres faz sentido, portanto, os operadores relacionais podem ser aplicados a caracteres. Os significados de "menor que" e "maior que" quando aplicados a caracteres são "vem antes" e "vem depois", respectivamente, no conjunto de caracteres. Assim, o caractere 'A' é menor que 'B', 'B' é menor que 'C' e assim por diante. Da mesma forma, o caractere '1' (não o número) é menor que '2', '2' é menor que '3' e assim por diante. Se você quiser comparar "A" com "1", deverá procurar a relação entre esses dois caracteres no conjunto de caracteres que estiver usando.

Booleano: Como dissemos na seção anterior, o tipo de dado booleano consiste em dois valores: TRUE e FALSE. Também podemos atribuir uma expressão booleana a uma variável booleana. Aqui está o programa de pares usando variáveis booleanas.

```
write "Quantidade de pares de entrada: "
read numeroDePares
numerosLidos ← 0
parar ← (numerosLidos == numeroDePares)
while (not parar)
   write "Entre com dois valores: "
   read numero1
   read numero2
   umMenor ← numero1 < numero2
   if (umMenor)
      print numero1, " ", numero2
   else
      print numero2, " ", numero1
   numerosLidos ← numerosLidos + 1
   parar ← (numerosLidos == numerosDePares)
```

Inteiros, reais, caracteres e booleanos são chamados de tipos de dados simples ou atômicos, porque cada valor é distinto e não pode ser subdividido em partes. No último capítulo, discutimos tipos de dados compostos – ou seja, tipos de dados compostos por uma coleção de valores. O tipo de dado string possui algumas das propriedades de um tipo composto, mas é frequentemente considerado um tipo de dado simples.

Strings: Uma string é uma sequência de caracteres que, em algumas linguagens, pode ser considerada como um valor de dado. Por exemplo,

```
"Isso é uma string."
```

é uma string contendo 18 caracteres: 1 letra maiúscula, 13 letras minúsculas, 3 espaços em branco e um ponto final. As operações definidas em strings variam de linguagem para linguagem, mas frequentemente incluem a concatenação de strings e a comparação de strings em termos de ordem lexicográfica. Outras linguagens fornecem uma matriz completa de operações, como obter uma substring de uma dada string ou pesquisar uma substring em uma dada string.

Observe que usamos aspas simples para delimitar caracteres e aspas duplas para delimitar strings. Algumas linguagens de alto nível usam o mesmo símbolo para ambos, não distinguindo entre um caractere e uma string com um único caractere.

Declarações

Uma declaração é uma instrução da linguagem que associa um identificador a uma variável, uma ação ou alguma outra entidade dentro da linguagem que pode receber um nome para que o programador possa se referir a esse item pelo nome. Nesta seção, discutiremos como uma variável é declarada. Posteriormente, veremos como as ações recebem nomes.

* Declaração: Uma instrução que associa um identificador a uma variável, uma ação ou alguma outra entidade dentro da linguagem que pode receber um nome para que o programador possa se referir a esse item pelo nome.

Linguagem	Declaração de variáveis
Python	`Não necessário`

VB.NET	`Dim sum As Single = 0.0F  'set up word with 0 as contents'`
	`Dim num1 As Integer  'set up a two byte block for num1'`
	`Dim num2 As Integer  'set up a two byte block for num2'`
	`Dim num3 As Integer  'set up a two byte block for num3'`
	`. . .`
	`Num1 = 1`

C++ ou Java	`float sum = 0.0;  'set up word with 0 as contents'`
	`int num1;  'set up a two byte block for num1'`
	`int num2;  'set up a two byte block for num2'`
	`int num3;  'set up a two byte block for num3'`
	`. . .`
	`Num1 = 1;`

Estes exemplos ilustram algumas diferenças entre linguagens de alto nível. Por exemplo, VB.NET usa uma palavra reservada para sinalizar uma declaração. Uma palavra reservada é uma palavra em uma linguagem que tem um significado especial; ela não pode ser usada como um identificador. **Dim** é uma palavra reservada em VB.NET usada para declarar variáveis. C++ e Java não usam uma palavra reservada para esse propósito.

* Palavra reservada: Uma palavra em uma linguagem que tem um significado especial; ela não pode ser usada como um identificador.

C++ e Java usam o ponto e vírgula para encerrar uma instrução na linguagem. VB.NET usa o fim de linha ou o símbolo de comentário para encerrar a instrução. Programas em Python não requerem declarações porque Python não é uma linguagem fortemente tipada. Python usa o sinal sharp (#) como o início de um comentário que se estende até o final da linha. Lembre-se de que o Pep/8 usa um ponto e vírgula para sinalizar que o que se segue é um comentário.

C++, Java, Python e VB.NET diferenciam maiúsculas de minúsculas, o que significa que duas cópias do mesmo identificador, quando capitalizadas de forma diferente, são consideradas palavras diferentes. Assim, Integer, INTEGER, InTeGeR e INTeger são considerados quatro identificadores diferentes em linguagens que diferenciam maiúsculas de minúsculas. C++, Java e VB.NET têm uma coleção de nomes de tipos para vários tamanhos de números inteiros e reais. Embora Python não declare identificadores, ele possui as palavras reservadas **long**, **int**, **float** e **bool**.

* Case sensitive: letras maiúsculas (uppercase) e minúsculas (lowercase) não são consideradas a mesma coisa; dois identificadores com a mesma grafia mas com capitalização diferente são considerados dois identificadores distintos.

Essas diferenças são importantes? Sim, se você estiver escrevendo um programa em uma dessas linguagens. No entanto, são apenas questões sintáticas – isto é, maneiras diferentes de fazer a mesma coisa. O conceito importante é que um identificador está associado a um lugar na memória e pode ou não estar associado a um tipo de dado. Nos exercícios, pedimos que você compare as diferenças sintáticas que surgem nestes exemplos.

O uso de letras maiúsculas e minúsculas em identificadores faz parte da cultura de uma linguagem. Em nossos exemplos, tentamos manter o estilo comum na cultura da linguagem. Por exemplo, a maioria dos programadores C++ começa nomes de variáveis com letras minúsculas e subprogramas com letras maiúsculas, enquanto programadores VB.NET tendem a começar nomes de variáveis com letras maiúsculas.

Estruturas de Entrada/Saída

Em nossos algoritmos de pseudocódigo, usamos as expressões **Read** e **Write** ou **Print** para indicar que estávamos interagindo com o ambiente externo ao programa. **Read** era para obter um valor de fora do programa e armazená-lo em uma variável dentro do programa, e **Write** e **Print** eram para exibir uma mensagem para o humano ver.

Linguagens de alto nível veem dados de texto como um fluxo de caracteres dividido em linhas. A forma como os caracteres são interpretados depende dos tipos de dados dos locais onde os valores devem ser armazenados. Qualquer instrução de entrada tem três partes: a declaração das variáveis nas quais os dados devem ser colocados, a instrução de entrada com os nomes das variáveis a serem lidas e o próprio fluxo de dados. Como exemplo, vejamos o algoritmo de pseudocódigo para inserir três valores:

```
read nome, idade, salarioHora
```

Em uma linguagem fortemente tipada, as variáveis **nome**, **idade** e **salarioHora** teriam que ser declaradas juntamente com seus respectivos tipos de dados. Vamos supor que os tipos sejam string, inteiro e real. A instrução de entrada listaria as três variáveis. O processamento prosseguiria da seguinte forma. O primeiro item de dados no fluxo de entrada seria considerado uma string, porque nome é do tipo string. A string seria lida e armazenada em **nome**. A próxima variável é um inteiro, portanto, a operação de leitura espera encontrar um inteiro no fluxo de entrada. Este valor é lido e armazenado em **idade**. A terceira variável é um número real, então a operação de leitura espera encontrar um valor real próximo no fluxo de entrada para ser armazenado como **salarioHora**.

O fluxo de entrada pode ser do teclado ou de um arquivo de dados, mas o processo é o mesmo: A ordem em que as variáveis são listadas na instrução de entrada deve ser a mesma que a ordem em que os valores ocorrem no fluxo de entrada. Os tipos das variáveis inseridas determinam como os caracteres no fluxo de entrada são interpretados. Ou seja, o fluxo de entrada é apenas uma série de caracteres ASCII (ou Unicode). O tipo da variável na qual o próximo valor deve ser armazenado determina como uma sequência de caracteres é interpretada. Para simplificar, vamos supor que a instrução de entrada considere que um espaço em branco separa cada valor de dados. Por exemplo, dado o fluxo de dados

```
Maggie 10 12.50
```

"Maggie" seria armazenado em **nome**, **10** seria armazenado em **idade** e **12.50** seria armazenado em **salarioHora**. Tanto **10** quanto **12.50** são lidos como caracteres e convertidos para os tipos inteiro e real, respectivamente.

Em uma linguagem que não é fortemente tipada, o formato da entrada determina o tipo. Se a entrada aparecer entre aspas, presume-se que seja uma string e é armazenada dessa forma. Se a entrada for um número, ela é armazenada dessa forma.

Instruções de saída criam fluxos de caracteres. Os itens listados na instrução de saída podem ser valores literais ou nomes de variáveis. Valores literais são números ou strings escritos explicitamente na instrução de saída (ou em qualquer instrução, nesse caso). Os valores a serem gerados são processados um de cada vez, observando o tipo do identificador ou literal. O tipo determina como o padrão de bits deve ser interpretado. Se o tipo for uma string, os caracteres são gravados no fluxo de saída. Se o padrão de bits for um número, o número é convertido nos caracteres que representam os dígitos e os caracteres são escritos.

Em uma linguagem fortemente tipada, independentemente da sintaxe das instruções de entrada/saída ou de onde os fluxos de entrada/saída estão, a chave para o processamento reside no tipo de dado que

determina como os caracteres devem ser convertidos em um padrão de bits (entrada) e como um padrão de bits deve ser convertido em caracteres (saída). Em uma linguagem que não é fortemente tipada, o formato da própria entrada determina como o padrão de bits deve ser convertido.

Aqui estão as instruções de entrada e saída nos quatro linguagens de programação que estamos usando para demonstrações. Os prompts foram omitidos. As instruções de entrada estão em preto; as instruções de saída estão em verde.

Linguagem	Instruções de entrada e saída
C++	`cin >> nome >> idade >> salarioHora;` `cout << nome << idade << salarioHora;`

Java	`Scanner inData;`
	`inData = new Scanner(system.in);`
	`nome = inData.nextLine();`
	`idade = inData.nextInt();`
	`salarioHora = inData.nextFloat();`
	`System.out.println(nome, ' ', idade, ' ', salarioHora);`

Python	`nome = input()`
	`idade = input()`
	`salarioHora = input()`
	`print(nome, idade, salarioHora)`

VB.NET	`Uses windowing`

Estruturas de Controle

Nosso pseudocódigo forneceu três maneiras de alterar o fluxo de controle do algoritmo: repetição, seleção e subprograma. Essas construções são chamadas de estruturas de controle porque determinam a ordem em que outras instruções em um programa são executadas.

* Estrutura de controle: Uma instrução que determina a ordem em que outras instruções em um programa são executadas.

No artigo seminal "Notes on Structured Programming" (ou Notas sobre Programação Estruturada), publicado em 1972, Edsger W. Dijkstra destacou que os programadores devem ser precisos e disciplinados – em outras palavras, devem usar apenas estruturas de controle selecionadas. Este artigo e os outros publicados com ele introduziram a era da **programação estruturada**.[10] De acordo com essa visão, cada unidade lógica de um programa deve ter apenas uma entrada e uma saída. O programa não deve entrar e sair aleatoriamente de módulos lógicos. Embora os programas possam ser projetados dessa maneira em linguagem assembly, usando instruções que se ramificam para outras partes do programa, as linguagens de programação de alto nível introduziram construções de controle que tornaram essa disciplina fácil de seguir. Essas construções são instruções de seleção, instruções de repetição e instruções de subprograma. Com essa abordagem, instruções de ramificação irrestritas não são mais necessárias.

Em nossos algoritmos de pseudocódigo, usamos recuo para agrupar instruções dentro do corpo de uma instrução **if** ou **while**. Python usa recuo, mas as outras linguagens usam marcadores reais. VB.NET usa **End If** e **End While** para encerrar as instruções correspondentes. Java e C++ usam chaves (**{ }**) como delimitadores.

As tabelas a seguir mostram segmentos de código usando instruções **if** e **while** nas linguagens de demonstração.

Linguagem	Instruções if

Python	`if temperatura > 75:`
	`  print("Não é necessário jaqueta.")`
	`else:`
	`  print("Uma jaqueta básica é necessário.")`
	`# agrupamento de marcas de identação`

VB.NET	`If (temperatura > 75) Then`
	`  MsgBox("Não é necessário jaqueta.")`
	`Else:`
	`  MsgBox("Uma jaqueta básica é necessário.")`
	`End If`

C++	`if (temperatura > 75)`
	`  cout << "Não é necessário jaqueta.";`
	`else`
	`  cout << "Uma jaqueta básica é necessário.";`

Java	`if (temperatura > 75)`
	`  System.out.println("Não é necessário jaqueta.");`
	`else`
	`  System.out.println("Uma jaqueta básica é necessário.";`

Linguagem	Instruções while controlada por contador de loop

Python	`count = 0`
	`while count < limite:`
	`  . . .`
	`  count = count + 1`
	`# identação marca o corpo do loop`

VB.NET	`Count = 0`
	`While (Count < Limite)`
	`  . . .`
	`  Count = Count + 1`
	`End While`

C++ e Java	`count = 1`
	`while (count <= limite) {`
	`  . . .`
	`  count = count + 1`

Edsger Dijkstra[11]



Cortesia de Staci Norman, UTCS, Universidade do Texas em Austin.

Cada campo da atividade humana tem seus principais colaboradores, aclamados por seus insights teóricos, extensões de ideias fundamentais ou mudanças inovadoras que redefiniram o assunto. Assim como Beethoven, Schubert, Mozart e Hayden soam verdadeiros no mundo da música clássica, e os Beatles, Rolling Stones e The Who se destacam no rock, Edsger Dijkstra tem um lugar reservado para ele no hall da fama da linguagem de computação.

Nascido em Roterdã, filho de um químico holandês, em 1930, Dijkstra cresceu com uma predileção formalista pelo mundo. Enquanto estudava na Universidade de Leiden, na Holanda, frequentou um curso de verão sobre programação em Cambridge, Inglaterra, e ficou fascinado por programação. Em 1952, conseguiu um emprego de meio período no Centro de Matemática de Amsterdã, onde continuou a trabalhar após se formar. Dijkstra chegou aos Estados Unidos no início da década de 1970 como pesquisador da Burroughs Corporation e, em setembro de 1984, ingressou na Universidade do Texas em Austin, onde ocupou a Cátedra Schlumberger Centennial em Ciências da Computação. Aposentou-se em novembro de 1999.

Uma das contribuições mais famosas de Dijkstra para a programação foi sua forte defesa dos princípios da programação estruturada. Dijkstra observou que programas escritos com instruções **goto** frequentemente se transformavam em um ninho de ratos, com saltos de um lado para o outro entre seções desorganizadas e ad-hoc de programas, tornando os programas difíceis de entender até mesmo para seus autores – sem mencionar os colegas que mais tarde poderiam ser solicitados a manter o programa. Dijkstra argumentou que o **goto** não era o princípio e o fim de todas as estruturas de controle e incentivou fortemente o uso de construções iterativas, ou de loop, que delimitam claramente o escopo da ramificação em um programa e efetivamente autodocumentam o programa. Dijkstra afirmou que aderir a esses princípios de programação estruturada tornaria os programas muito mais fáceis de entender e manter, e menos propensos a conter erros.

Além de suas claras contribuições teóricas, Dijkstra é uma figura interessante no mundo da computação. Ele desenvolveu a reputação de expressar sua opinião, muitas vezes de maneiras inflamadas ou dramáticas que a maioria de nós não consegue evitar. Por exemplo, Dijkstra certa vez observou que "o uso do COBOL

paralisa a mente; seu ensino deve, portanto, ser considerado uma ofensa criminal". Não sendo alguém que destaca apenas uma linguagem para suas críticas, ele também disse que "é praticamente impossível ensinar boa programação a alunos que já tiveram contato com BASIC; como programadores em potencial, eles são mentalmente mutilados, sem esperança de regeneração". Algumas pessoas consideram sua mensagem convincente e acreditam que sua maneira é politicamente necessária para expressar seu ponto de vista. Outros, cientes do desenvolvimento histórico das linguagens e dos contextos em que foram projetadas, apreciam sua mensagem, mas consideram sua maneira um tanto estridente.

Além de seu trabalho em design de linguagens, Dijkstra é conhecido por seu trabalho em provas de correção de programas. A área de correção de programas é uma aplicação da matemática à programação de computadores. Pesquisadores estão tentando construir uma linguagem e uma técnica de prova que possam ser usadas para certificar incondicionalmente que um programa funcionará de acordo com suas especificações – totalmente livre de bugs. Desnecessário dizer que, seja sua aplicação em sistemas de faturamento de clientes ou de controle de voo, essa afirmação seria extremamente valiosa.

Em 1972, a Association for Computing Machinery reconheceu as ricas contribuições de Dijkstra para a área, concedendo-lhe o distinto Prêmio Turing. A menção ao prêmio dizia:

> Edsger Dijkstra foi um dos principais contribuidores, no final da década de 1950, para o desenvolvimento do ALGOL, uma linguagem de programação de alto nível que se tornou um modelo de clareza e rigor matemático. Ele é um dos principais expoentes da ciência e da arte das linguagens de programação em geral e contribuiu significativamente para a nossa compreensão de sua estrutura, representação e implementação. Seus quinze anos de publicações abrangem desde artigos teóricos sobre teoria dos grafos até manuais básicos, textos expositivos e contemplações filosóficas no campo das linguagens de programação.

Em 1989, o Special Interest Group for Computer Science Education – SIGCSE (ou Grupo de Interesse Especial para o Ensino de Ciência da Computação) o homenageou com o prêmio de Contribuições Excepcionais para o Ensino de Ciência da Computação.

Dijkstra e sua esposa retornaram à Holanda quando ele descobriu que tinha apenas meses de vida. Ele sempre disse que queria se aposentar em Austin, Texas, mas morrer na Holanda. Dijkstra faleceu em 6 de agosto de 2002.

Em março de 2003, o seguinte e-mail foi enviado à comunidade de computação distribuída:

> Anunciamos que o prêmio anteriormente conhecido como "Prêmio de Artigo Influente do PODC" foi renomeado para "Prêmio Edsger W. Dijkstra em Computação Distribuída", em homenagem ao falecido Edsger W. Dijkstra, um pioneiro na área de computação distribuída. Seu trabalho fundamental sobre primitivas de concorrência (como o semáforo), problemas de concorrência (como exclusão mútua e deadlock), raciocínio sobre sistemas concorrentes e autoestabilização constitui um dos pilares mais importantes sobre os quais o campo da computação distribuída se baseia. Nenhum outro indivíduo teve maior influência na pesquisa em princípios de computação distribuída.

As informações sobre o prêmio podem ser encontradas em www.podc.org/dijkstra/.

Dijkstra era conhecido por muitas frases concisas e espirituosas. Uma que todos que estudam computação devem ponderar é que "a ciência da computação não tem mais a ver com computadores do que a astronomia tem a ver com telescópios".

A tabela a seguir mostra como as linguagens VB.NET e C++ definem um subprograma que não retorna um único valor. Neste exemplo, há dois parâmetros de valor inteiro e um parâmetro de referência real. Novamente, esta ilustração tem como objetivo dar uma ideia da rica variedade de sintaxe que abunda em linguagens de alto nível, não para torná-lo competente na escrita desta construção em qualquer uma delas. O "e" comercial (**&**) usado em C++ não é um erro tipográfico; ele sinaliza que **three** é um parâmetro de referência.

Linguagem	Declaração de subprogramas

VB.NET	`Public Sub Example( ByVal one As Integer, ByVal two As Integer,`
	`  ByRef three As Single)`
	`  . . .`
	`End Sub`

C++ e Java	`void Example (int one, int two, float& thress) {`
	`  . . .`
	`}`

Não mostramos um exemplo em Java ou Python porque eles lidam com a memória de forma muito diferente, permitindo apenas parâmetros de valor.

Lógica Aninhada

As instruções a serem executadas ou ignoradas em qualquer instrução de controle podem ser instruções simples ou blocos (instruções compostas) – não há restrição sobre o que essas instruções podem ser. De fato, a instrução a ser ignorada ou repetida pode conter uma estrutura de controle. Instruções de seleção podem ser aninhadas em estruturas de repetição; estruturas de repetição podem ser aninhadas em instruções de seleção. Instruções de seleção e repetição podem ser aninhadas dentro de subprogramas, e chamadas de subprogramas podem ser aninhadas dentro de estruturas de repetição ou seleção.

Abordamos o aninhamento em nossos algoritmos, mas o tópico merece uma nova análise. Veja, por exemplo, o algoritmo que conta e soma dez números positivos em um arquivo:

```
soma ← 0                        // inicializa soma
posCont ← 0                     // Inicializa posCont
while (posCont < 10)            // Testa posCont
   read valor
   if (valor > 0)               // Testa para verificar se atualiza posCont
      posCont ← posCont + 1     // Atualiza posCont
      soma ← soma + valor       // Adiciona valor a soma
// comandos fora do loop
```

A estrutura de controle de seleção está inserida em uma estrutura de repetição. Se quiséssemos somar e imprimir os valores semanais de precipitação pluvial de um ano, teríamos as seguintes estruturas em repetição aninhadas:

```
contaSemana ← 1
while (contaSemana <= 52)
   somaSemana ← 0
   contaDias ← 1
   while (contaDias <= 7)
      read choveu
      somaSemana ← somaSemana + choveu
      contaDias ← contaDias + 1
   write "Semana ", contaSemana, "total: ", somaSemana
   contaSemana ← contaSemana + 1
```

Estruturas de controle dentro de estruturas de controle dentro de estruturas de controle... Teoricamente, não há limite para a profundidade do aninhamento das estruturas de controle! No entanto, se o aninhamento se tornar muito difícil de acompanhar, você deve dar um nome à tarefa aninhada e torná-la um subprograma, apresentando sua implementação posteriormente. Por exemplo, examine a versão alternativa do algoritmo de pseudocódigo anterior. Qual é mais fácil de acompanhar?

? Programadores devem ler filosofia?

"Programação de computadores e design de software são essencialmente filosofia altamente estruturada que aborda um aspecto particular do mundo. Filósofos se beneficiariam da leitura de alguns livros sobre programação, e programadores se beneficiariam da leitura de alguns livros sobre filosofia."[12]

```
contaSemana ← 1
while (contaSemana <= 52)
   somaSemana ← CalculaSomaSemana(contaSemana)
   write "Semana ", contaSemana, "total: ", somaSemana
   contaSemana ← contaSemana + 1
```

```
CalculaSomaSemana(contaSemana)
   somaSemana ← 0
   contaDias ← 1
   while (contaDias <= 7)
      read choveu
      somaSemana ← somaSemana + choveu
      contaDias ← contaDias + 1
   return somaSemana
```

Processamento Assíncrono

Você provavelmente cresceu usando uma interface gráfica de usuário (GUI) que depende do uso de um mouse para manipular múltiplas janelas em uma tela. Clicar se tornou uma importante forma de entrada no computador. De fato, para muitas aplicações, preencher caixas e clicar em botões para dizer que a entrada está pronta tornou-se a única forma de entrada. No processamento de fluxo tradicional, uma instrução de entrada é executada na sequência em que é encontrada. Aqui estão as quatro primeiras instruções do algoritmo mostrado anteriormente:

```
write "Quantidade de pares de entrada: "
read numeroDePares
numerosLidos ← 0
while (numerosLidos < numeroDePares)
   ...
```

Esperamos que essas instruções sejam executadas em sequência. A saída é gravada em uma janela, um valor é lido do fluxo de entrada, outro valor é armazenado e a repetição **while** é executada. A entrada e a saída do fluxo estão dentro do fluxo sequencial do programa. O clique do mouse, por outro lado, não ocorre dentro da sequência do programa. Isto é, um usuário pode clicar com o mouse a qualquer momento durante a execução de um programa. O programa deve reconhecer quando um clique do mouse ocorreu, processá-lo e então continuar. Esse tipo de processamento é chamado de assíncrono, o que significa "não ao mesmo tempo". O mouse pode ser clicado a qualquer momento; ele não é sincronizado com nenhuma outra instrução.

* Assíncrono: Não ocorre ao mesmo tempo que alguma operação específica do computador; em outras palavras, não é sincronizado com as ações do programa.

Processamento assíncrono também é chamado de processamento orientado a eventos. Em outras palavras, o processamento está sob o controle de eventos que ocorrem fora da sequência de instruções do programa. O processamento assíncrono é usado com frequência em Java e VB.NET, mas com menos frequência em outras linguagens.

9.5 Funcionalidade de Linguagens Orientadas a Objetos

Como você pode imaginar pela discussão anterior sobre projeto orientado a objetos, a construção básica em uma linguagem orientada a objetos é a **classe**. Além de analisar a construção de classe nesta seção, examinamos os três ingredientes essenciais em uma linguagem orientada a objetos: **encapsulamento**, **herança** e **polimorfismo**. Esses ingredientes promovem a reutilização, reduzindo assim o custo de construção e manutenção de software.

Encapsulamento

No Capítulo 7, falamos sobre threads importantes que permeiam a discussão sobre resolução de problemas. Duas dessas threads eram **abstração e ocultação de informações**. Lembre-se de que ocultação de informações é a prática de ocultar os detalhes de um módulo com o objetivo de controlar o acesso a eles. Dissemos no Capítulo 7 que abstração era um modelo de um sistema complexo que inclui apenas os detalhes essenciais para o observador. Definimos três tipos de abstração, mas as definições de cada um começavam e terminavam com as palavras "A separação da visão lógica de ... de seus detalhes de implementação". Abstração é o objetivo; ocultação de informações é uma técnica usada para atingir o objetivo.

Na discussão sobre design orientado a objetos, dissemos que encapsulamento é o agrupamento de dados e ações de tal forma que as propriedades lógicas dos dados e ações permaneçam separadas dos detalhes de implementação. Outra maneira de dizer isso é que encapsulamento é um recurso da linguagem que impõe a ocultação de informações. A implementação de um módulo é oculta em um bloco separado com uma interface formalmente especificada. Um objeto sabe coisas sobre si mesmo, mas não sobre qualquer outro objeto. Se um objeto precisa de informações sobre outro objeto, ele deve solicitar essas informações daquele objeto.

* Encapsulamento (segunda definição): Um recurso da linguagem que impõe a ocultação de informações.

A construção usada para fornecer encapsulamento é a classe. Assim como o conceito de classe domina o design orientado a objetos, o conceito de classe é o principal recurso do Java e de outras linguagens orientadas a objetos. Infelizmente, as definições relacionadas não são padronizadas entre as fases de design e implementação. Na fase de design (resolução de problemas), um objeto é uma coisa ou entidade que faz sentido dentro do contexto do problema. Na fase de implementação, uma classe é uma construção de linguagem que é um padrão para um objeto e fornece um mecanismo para encapsular as propriedades e ações da classe de objeto.

* Objeto (fase de resolução de problemas): Uma entidade ou coisa que é relevante no contexto de um problema.

* Classe (fase de implementação): Um padrão para um objeto.

* Classe de objeto (classe) (fase de resolução de problemas): Uma descrição de um grupo de objetos com propriedades e comportamentos semelhantes.

* Objeto (fase de implementação): Uma instância de uma classe.

Classes

Sintaticamente, uma classe é como um registro, pois é um tipo de dado composto heterogêneo. No entanto, os registros têm sido tradicionalmente considerados estruturas passivas; somente nos últimos anos eles passaram a ter subprogramas como campos. A classe, por outro lado, é uma estrutura ativa e quase sempre possui subprogramas como campos. A única maneira de manipular os campos de dados de uma classe é por meio dos métodos (subprogramas) definidos na classe.

Eis como poderíamos definir a classe **Person**, com base em uma desenvolvida anteriormente:

```
public class Person
   // Variáveis de classe
   Name name
   String telephone
   String email

   // Métodos de classe
   Initialize()
      // Código para Inicialize

   public Print()
      // Código para Print

   public Name GetName()
      return name

   public String GetEmail()
      return email

   public String GetTelephone()
      return telephone
```

A FIGURA 9.4 visualiza a classe **Person**. Os campos de variáveis estão em branco. Os campos de subprograma estão em laranja para indicar que não contêm valores, mas são subprogramas.

Em nossos algoritmos, usamos identificadores para variáveis simples e arrays sem nos preocuparmos com sua origem. Se usarmos um identificador para representar uma classe, no entanto, devemos solicitar explicitamente que a classe seja criada antes de podermos usá-la. Ou seja, temos que instanciar a classe usando o operador **new** para obter um objeto que se encaixe no padrão. Este operador recebe o nome da classe e retorna uma instância da classe. Este algoritmo instancia uma classe **Person**, obtém um objeto **aPerson** da classe e armazena e recupera valores no objeto. Primeiro, instanciamos um objeto **Name**, mas assumimos que as variáveis de string **email** e **telephone** já possuem valores.

* Instanciar: Para criar um objeto a partir de uma classe.



FIGURA 9.4 Classe Person.

```
Name aName = new Name()
aName.Initialize("Frank", "Jones")
Person aPerson = new Person()
aPerson.Initialize(aName, telefone, email)
aPerson.Print()
write "Name: ", aPerson.GetName().Print()
write "Telephone: ", aPerson.GetTelephone()
write "Email: ", aPerson.GetEmail()
```

Os algoritmos que declaram objetos de classe podem acessar os campos da classe apenas por meio dos subprogramas (chamados métodos) da classe.

Os campos em uma classe são privados (**private**) por padrão. Ou seja, nenhum dos campos, seja de dados ou método, de um objeto de uma classe específica pode ser acessado por qualquer outro objeto, a menos que o campo esteja marcado como público (**public**). Se uma classe precisar disponibilizar um método para ser chamado por um objeto de outra classe, a classe deve especificar explicitamente que o método é público. Os métodos da classe **Person** são marcados como públicos para que um programa que os utilize possa chamá-los.

Herança

Herança é uma propriedade das linguagens orientadas a objetos nas quais classes podem herdar dados e métodos de outras classes. Esse relacionamento é um relacionamento "é um". Uma **superclasse** é uma classe da qual se herda; uma **classe derivada** é uma classe que herda. As classes formam uma hierarquia de herança. Na hierarquia, os objetos se tornam mais especializados quanto mais abaixo na hierarquia nos movemos. Classes mais abaixo na hierarquia herdam todos os comportamentos e dados de sua superclasse pai.

* Herança: Um mecanismo pelo qual uma classe adquire as propriedades – campos de dados e métodos – de outra classe.

Em nosso exemplo mais recente, trabalhamos com a classe **Person**. Em uma linguagem orientada a objetos, podemos definir uma classe **Student**, que herda todas as propriedades da classe **Person** e adiciona mais campos de dados para armazenar endereço (**address**) e o número de telefone (**telephone**). Objetos da classe **Person** têm apenas um endereço e número de telefone, mas objetos da classe **Student** têm dois: um herdado da classe **Person** e um definido dentro da classe **Student**. Dizemos que a classe **Student** é derivada da classe **Person**.

Aqui estão os cabeçalhos dos cartões CRC para **Person** e **Student**. Observe que os campos Subclasse e Superclasse nos cartões CRC foram preenchidos.



Vamos supor que definimos as classes **Person** e **Student**. O seguinte algoritmo instancia as classes **Person** e **Student** e as manipula:

```
Person myPerson = new Person()      // Instancia myPerson
Student myStudent = new Student()   // Instancia myStudent
myPerson.Initialize(...)            // myPerson se inicializa
myStudent.Initialize(...)           // myStudent se inicializa
myPerson.Print()                    // myPerson se imprime
myStudent.Print()                   // myStudent se imprime
```

A herança promove a reutilização, permitindo que um aplicativo pegue uma classe já testada e derive dela uma classe que herda as propriedades de que o aplicativo precisa. Outras propriedades e métodos necessários podem então ser adicionados à classe derivada.

? Denunciar pirataria de software

Se você souber de alguma ocorrência de pirataria de software, preencha um relatório online e você poderá se qualificar para uma recompensa. Acesse www.bsa.org e clique em "Report Piracy".

Polimorfismo

Na seção anterior, as classes **Person** e **Student** possuem métodos chamados **Print** e **Initialize**. O método da classe **Person** imprime o endereço definido em sua classe, e o método da classe **Student** imprime o endereço definido em sua classe. Aqui, temos dois métodos com o mesmo nome, mas implementações diferentes. A capacidade de uma linguagem de programação de lidar com essa aparente ambiguidade é chamada de **polimorfismo**. Como a linguagem sabe qual método se refere quando **Initialize** ou **Print** é invocado pela unidade chamadora? Métodos que fazem parte de uma classe são aplicados a uma instância da classe pela unidade chamadora. A classe do objeto ao qual o método é aplicado determina quais versões do método **Initialize** ou **Print** serão usadas.

* Polimorfismo: A capacidade de uma linguagem de determinar, em tempo de execução, qual dos vários métodos possíveis será executado para uma determinada invocação.

Por exemplo, se tivéssemos **jane** como uma instância da classe **Person** e **jack** como uma instância da classe **Student**, **jane.Print** invocaria o método definido na classe **Person** para imprimir as informações de **jane**. E **jack.Print** invocaria o método definido na classe **Student** para imprimir as informações de **jack**. A classe **Student** poderia adicionar um método **PrintHomeAddress**, que imprimiria o endereço residencial de um aluno.

A combinação de herança e polimorfismo permite ao programador construir hierarquias úteis de classes que podem ser reutilizadas em diferentes aplicações. A reutilização não se aplica apenas a linguagens orientadas a objetos; no entanto, a funcionalidade das linguagens orientadas a objetos facilita a escrita de seções gerais e reutilizáveis de código.

9.6 Comparação entre Projetos Procedurais e Orientados a Objetos

No final do Capítulo 8, utilizamos a implementação dos algoritmos ADT List para descrever o processo de subprogramas com e sem retorno de valor. A implementação do ADT List era uma lista de variáveis de registro, que continha um array de valores e um campo de comprimento, que era passada aos algoritmos List para serem processados. O programa chamador definia a implementação da lista e escrevia os algoritmos para manipulá-la. Os subprogramas eram tarefas do programa que precisavam de uma lista.

Em um projeto orientado a objetos, a estrutura de dados List e os subprogramas seriam vinculados em uma classe da seguinte maneira:

```
public class List
   // Variáveis de classe
   values[]
   length

   // Métodos de classe
   public Boolean IsThere(item)
      position ← 0
      while (position < length AND values[position] != item)
         position ← position + 1
      return position < length

   public Delete(item)
      position ← 1
      while (values[position] != item)
         position ← position + 1
      Swap(values[length - 1], values[position])
      length ← length – 1

   // Restante das operações da classe
```

O código dos métodos tem acesso direto às variáveis da classe; o código do usuário não. A classe seria compilada separadamente e o programa que quisesse usá-la a incluiria em seu programa. Aqui está um segmento de pseudocódigo que manipularia um objeto List:

```
List list = new List()        // Instância de um objeto List
while (more values)
   read aValue
   if (NOT list.IsThere(aValue))
      list.insert(aValue)
write "Informe o valor para remover (ou digite Sair): "
read aValue
if (aValue != "Sair")
   if (list.IsThere(aValue))
      list.Delete(aValue)
```

Na versão procedural, a lista é representada por um registro que é passado aos subprogramas que operam sobre ela. A estrutura de dados e os subprogramas que a manipulam fazem parte do programa do usuário. Na versão orientada a objetos, a implementação de uma classe objeto é ocultada do usuário por meio de encapsulamento.

RESUMO

O projeto orientado a objetos concentra-se em determinar os objetos dentro de um problema e abstrair (agrupar) esses objetos em classes com base em propriedades e comportamentos semelhantes.

Existem quatro estágios na decomposição orientada a objetos:

- Brainstorming, no qual fazemos uma primeira tentativa de determinar as classes no problem;
- Filtragem, na qual revisamos as classes propostas;
- Cenários, nos quais as responsabilidades de cada classe são determinadas; e
- Algoritmos de responsabilidade, nos quais os algoritmos são escritos para cada uma das responsabilidades.

Um tradutor assembly (ou montador) traduz um programa em linguagem assembly para código de máquina. Um compilador traduz um programa escrito em uma linguagem de alto nível para linguagem assembly (para ser posteriormente traduzido para código de máquina) ou para código de máquina. Um interpretador é um programa que traduz as instruções de um programa e as executa imediatamente. Um interpretador não gera código em linguagem de máquina.

Existem vários modelos de linguagens de programação de alto nível, classificados como imperativos (procedimentais e orientados a objetos) ou declarativos (funcionais e lógicos). O modelo imperativo descreve o processamento a ser realizado. O modelo declarativo descreve o que deve ser feito, não como deve ser realizado. O modelo procedural é baseado no conceito de uma hierarquia de tarefas a serem concluídas; o modelo orientado a objetos é baseado no conceito de objetos interagindo. O modelo funcional é baseado no conceito matemático de uma função; o modelo lógico é baseado na lógica matemática.

Uma expressão booleana é uma afirmação sobre o estado de um programa. Expressões booleanas são usadas para permitir que um programa execute uma seção de código ou outra (instruções condicionais) e repita uma seção de código (instruções de loop).

Cada variável em um programa é um determinado tipo de dado. Tipagem forte significa que as variáveis recebem um tipo e que apenas valores desse tipo de dado podem ser armazenados na variável. Armazenar um valor em uma variável é chamado de atribuição do valor à variável (instruções de atribuição).

Programas orientados a objetos são caracterizados pelas seguintes construções:

- Encapsulamento, um recurso da linguagem que impõe a ocultação de informações, implementado usando a construção de classe.
- Herança, um recurso da linguagem que permite que uma classe herde as propriedades e comportamentos de outra classe.
- Polimorfismo, a capacidade de uma linguagem de desambiguar entre operações com o mesmo nome.

EXERCÍCIOS

Para os Exercícios de 1 a 10, combine a atividade com a fase da metodologia orientada a objetos.

A) Brainstorming B) Filtragem C) Cenários D) Algoritmos de responsabilidade

1. Revisar uma lista de classes possíveis, procurando por classes duplicadas ou ausentes.
2. Fazer perguntas do tipo "e se ...".

3. Atribuir responsabilidades às classes.
4. Gerar uma primeira aproximação para a lista de classes em um problema.
5. Atribuir colaboradores a uma responsabilidade.
6. Desenvolver algoritmos para as responsabilidades listadas em um cartão CRC.
7. A saída desta fase é um cartão CRC totalmente desenvolvido para todas as classes.
8. A saída desta fase é o projeto pronto para ser traduzido em um programa.
9. Durante esta fase, relacionamentos de herança são estabelecidos.
10. Fase na qual as técnicas de programação funcional são apropriadas.

Para os Exercícios de 11 a 24, combine a questão com o sistema de tradução ou execução apropriado.

A) Interpretador B) Montador C) Compilador D) Código de máquina

11. O que traduz uma linguagem de alto nível para código de máquina?
12. O que traduz um programa Java para Bytecode?
13. O que executa Bytecode?
14. O que traduz um programa em linguagem assembly?
15. Qual é a saída de um montador?
16. O que recebe a entrada em uma linguagem de alto nível e direciona o computador para executar as ações especificadas em cada instrução?
17. O que executa a Máquina Virtual Java?
18. O que é usado para traduzir um programa em ALGOL?
19. O que é usado para traduzir um programa em APL?
20. O que é usado para traduzir um programa em COBOL?
21. O que é usado para traduzir um programa em FORTRAN?
22. O que é usado para traduzir um programa em Lisp?
23. O que é usado para traduzir um programa em Prolog?
24. Qual tradutor é executado mais lentamente?

Para os Exercícios de 25 a 46, combine o paradigma da linguagem e a linguagem ou a descrição da linguagem.

A) Procedural B) Funcional C) Lógica D) Orientada a Objetos

E) Linguagem procedural com características de orientação a objetos.

F) Linguagem orientada a objetos com algumas características procedural.

25. Qual paradigma descreve o FORTRAN com mais precisão?
26. Qual paradigma descreve o C++ com mais precisão?
27. Qual paradigma descreve o PASCAL com mais precisão?
28. Qual paradigma descreve o Java com mais precisão?
29. Qual paradigma descreve o Lisp com mais precisão?
30. Qual paradigma descreve o BASIC com mais precisão?
31. Qual paradigma descreve o Prolog com mais precisão?
32. Qual paradigma descreve o SIMULA com mais precisão?
33. Qual paradigma descreve o ALGOL com mais precisão?
34. Qual paradigma descreve o ML com mais precisão?
35. Qual paradigma descreve o Scheme com mais precisão?
36. Qual paradigma descreve o Python com mais precisão?
37. Qual paradigma descreve o C com mais precisão?
38. Qual paradigma descreve o Smalltalk com mais precisão?

39. As linguagens dominantes usadas na indústria ao longo da história da computação decorrem de qual paradigma?
40. Qual paradigma os pesquisadores japoneses escolheram para a quinta geração de computadores?
41. Qual paradigma permite ao programador expressar algoritmos como uma hierarquia de objetos?
42. Qual paradigma permite ao programador expressar algoritmos como uma hierarquia de tarefas?
43. Qual paradigma permite ao programador expressar algoritmos como funções matemáticas?
44. Qual paradigma não possui declaração de atribuição?
45. Qual paradigma usa recursão exclusivamente para expressar repetição?
46. Qual paradigma não possui variáveis?

Os exercícios de 47 a 84 são problemas ou perguntas de resposta curta.

47. Qual é a marca registrada de uma linguagem assembly?
48. Distinga entre um montador e um compilador.
49. Distinga entre um compilador e um interpretador.
50. Compare e contraste um montador, um compilador e um interpretador.
51. Descreva a portabilidade proporcionada por um compilador.
52. Descreva a portabilidade proporcionada pelo uso de Bytecode.
53. Descreva o processo de compilação e execução de um programa Java.
54. Discuta a palavra paradigma em sua relação com a computação.
55. Distinga entre paradigmas imperativos e declarativos.
56. Quais são as características do paradigma imperativo?
57. Quais são as características do paradigma funcional?
58. Quais são as características do paradigma lógico?
59. Quais são as características de um paradigma declarativo?
60. Como você faz perguntas em uma linguagem de programação?
61. O que é uma variável booleana?
62. O que é uma expressão booleana?
63. Dadas as variáveis **um**, **dois** e **três**, escreva uma asserção para cada uma das seguintes perguntas.
    a) Um é maior que dois e três?
    b) Um é maior que dois, mas menor que três?
    c) Todas as três variáveis são maiores que zero?
    d) Um é menor que dois ou um é menor que três?
    e) Dois é maior que um e três é menor que dois?
64. Escreva a tabela de operações para a operação booleana AND.
65. Escreva a tabela de operações para a operação booleana OR.
66. Escreva a tabela de operações para a operação booleana NOT.
67. O que é um tipo de dado?
68. O que é tipagem forte?
69. Defina os seguintes tipos de dados.
    a) Inteiro b) Real c) Caractere d) Booleano
70. O tipo de dado string é um tipo de dado atômico? Justifique sua resposta.
71. Se o mesmo símbolo for usado tanto para caracteres simples quanto para strings, como você pode distinguir entre um caractere simples e uma string de um caractere?
72. O que é uma declaração?
73. Preencha a tabela a seguir mostrando o marcador sintático apropriado ou palavra reservada para a linguagem apresentada, com base em suas observações das tabelas deste capítulo.

74. Como as diretivas de montagem .WORD e .BLOCK na linguagem de montagem Pep/8 diferem das declarações em linguagens de alto nível?
75. Distinga entre instruções a serem traduzidas e instruções para o programa tradutor.
76. Considere os seguintes identificadores: Address, ADDRESS, AddRess, Name, NAME, NomE.
    a) Quantos identificadores diferentes são representados se a linguagem for Python?
    b) Quantos identificadores diferentes são representados se a linguagem for VB .NET?
    c) Quantos identificadores diferentes são representados se a linguagem for C++ ou Java?

Linguagem	**Python**	**VB.NET**	**C++**	**Java**
Comentários				
Fim do comando				
Declaração de atribuição				
Tipo de dado real				
Tipo de dado inteiro				
Início da(s) declaração(ões)				

77. Distinga entre a definição de um objeto na fase de projeto e na fase de implementação.
78. Distinga entre a definição de uma classe na fase de projeto e na fase de implementação.
79. Distinga entre um campo e um método.
80. Como os objetos podem se relacionar?
81. Discuta as diferenças entre um projeto top-down e um projeto orientado a objetos.
82. Neste capítulo, delineamos uma estratégia para desenvolver uma decomposição orientada a objetos.
    a) Liste os quatro estágios.
    b) Descreva as características de cada estágio.
    c) Qual é o resultado de cada um dos quatro estágios?
    d) Cada um dos estágios é independente? Explique.
83. Projete os cartões CRC para um sistema de inventário para uma concessionária de veículos, utilizando brainstorming, filtragem e cenários.
84. Projete os cartões CRC para um banco de dados para um zoológico, utilizando brainstorming, filtragem e cenários.

QUESTÕES PARA REFLEXÃO

1. Diferencie um programa que a CPU pode executar diretamente de um programa que precisa ser traduzido.
2. Tanto o design top-down quanto o design orientado a objetos criam andaimes que são usados para escrever um programa. Todo esse andaime é apenas um desperdício de esforço? Ele é usado novamente? Qual o seu valor depois que o programa está em execução?
3. Qual das estratégias de resolução de problemas você mais usa? Você consegue pensar em algumas outras que você usa? Elas seriam apropriadas para a resolução de problemas de computação?
4. Você já foi enganado por uma farsa? Ficou com raiva ou apenas irritado?
5. Você ou alguém que você conhece já foi vítima de um golpista?
6. Você já recebeu um e-mail semelhante ao mostrado neste capítulo? Você respondeu?